

APPLICATION AUDIT REPORT

Project	Cake Design Academy — E-Learning Platform
Audit Type	Full-Stack · Frontend · Backend · Security · UX · Performance
Scope	React 19 Frontend + Spring Boot 3.2 Backend + PostgreSQL + Mux + PayZone
Date	March 31, 2026
Classification	CONFIDENTIAL
Prepared By	Senior QA Engineer / Security Analyst
Version	1.0.0



■ TABLE OF CONTENTS

1.	Executive Summary	Application health score and key findings
2.	Application Overview	Architecture, technologies, and structure
3.	Feature Audit Table	Status of all 47 features across all modules
4.	Detailed Bug Reports	20 individual bug reports with reproduction steps
5.	Backend Connectivity Report	Frontend-API endpoint mapping and mismatches
6.	UI/UX Issues	Interface and user experience findings
7.	Performance Analysis	Load, rendering, and optimization issues
8.	Security Risk Assessment	Vulnerabilities, exposures, and OWASP findings
9.	Recommendations & Fix Priority Plan	Roadmap ordered by business impact

1. EXECUTIVE SUMMARY

Overall Health Score: 62 / 100

The Cake Design Academy platform is a feature-rich e-learning application with a solid architectural foundation. The backend (Spring Boot 3.2) is well-structured with 200+ endpoints, 31 entities, and clean separation of concerns. The frontend (React 19 + TypeScript) follows modern patterns with Redux Toolkit and role-based routing. However, the audit identified **8 critical issues** (including exposed secrets in source code), **12 high-severity bugs**, and several features that are either incomplete, broken, or not connected to the backend. Immediate action is required for security vulnerabilities before any production deployment.

Category	Count	Key Finding
Total Features Audited	47	Across 9 functional modules
Working	31	66% of features fully operational
Partial	9	Fail at final step or partially connected
Broken / Not Connected	7	Missing frontend or backend integration
Critical Security Issues	8	Must fix BEFORE production
High Severity Bugs	12	Fix within sprint 1
Medium / Low Issues	15+	Fix within sprint 2-3
REST Endpoints (Backend)	200+	19 controllers, 27 repositories
API Services (Frontend)	60+	13 service files, 7 Redux slices
Database Migrations	6	V1–V6, 31 entities, fully mapped

Key Risk Areas:

- **CRITICAL:** Gmail SMTP app password hardcoded in application.yml — revoke and rotate immediately.
- **CRITICAL:** JWT secret key committed to source control — must be rotated.
- **CRITICAL:** CORS allows all origins with credentials=true — potential CSRF/CORS exploit.
- **CRITICAL:** Spring Boot Actuator endpoints publicly accessible without authentication.
- **HIGH:** 6 instructor dashboard routes all incorrectly mapped to /instructor/instructor-withdraw.
- **HIGH:** Community module fully implemented in backend but has zero frontend pages.
- **HIGH:** Real-time messaging (WebSocket) defined in environment but not implemented client-side.
- **HIGH:** Build artifacts and TypeScript error files committed to git repository.

2. APPLICATION OVERVIEW

2.1 Architecture Summary

Cake Design Academy is a full-stack e-learning platform targeting cake design education. It follows a classic client-server architecture with a REST API backend and a single-page React frontend. Authentication uses stateless JWT tokens stored in localStorage, with a 15-minute access token and 7-day refresh token. Video delivery is handled by Mux (with local filesystem fallback), payments by PayZone (Moroccan payment gateway), and transactional emails via Gmail SMTP.

Layer	Technology	Version	Notes
Frontend Framework	React	19.0.0	Create React App · TypeScript
State Management	Redux Toolkit	2.5.0	7 slices (auth, courses, student, instructor, admin, sidebar, theme)
UI Library	Ant Design + Bootstrap	5.23 / 5.3.3	Mixed usage · Luxury SCSS theme overlay
HTTP Client	Axios	1.13.4	Auto-refresh interceptor · 30s timeout
Routing	React Router DOM	7.1.1	Role-based guards (STUDENT / INSTRUCTOR / ADMIN)
Backend Framework	Spring Boot	3.2.3	Java 17 · Maven build
Security	Spring Security + JWT	HS256	15min access / 7day refresh token
ORM / DB	Spring Data JPA + Flyway	Hibernate 6	PostgreSQL 15 · 31 entities · 6 migrations
Video Hosting	Mux	API v1	Signed playback URLs · Direct upload · Local fallback
Payment Gateway	PayZone	REST API	HMAC-SHA256 signatures · Moroccan gateway
Email	Spring Mail	SMTP	Gmail SMTP · 4 HTML templates
Real-time	Spring WebSocket	STOMP	Configured but NOT implemented client-side
API Docs	SpringDoc OpenAPI	3.0	Swagger UI at /swagger-ui.html

2.2 Module Structure

Module	Endpoints	Capabilities
Authentication	11 endpoints	Login, Register, JWT, Email Verify, Password Reset
Course Management	24 endpoints	CRUD, Modules, Lessons, Mux Video, Curriculum
Enrollment & Progress	6 endpoints	Enroll, Lesson Progress, Completion
Payments	5 endpoints	PayZone integration, Webhooks, History
Subscriptions	5 endpoints	Yearly plan, Cancel, PayZone webhook
Quiz System	14 endpoints	Create/Publish quizzes, Student attempts, Scoring
Assignments	4 endpoints	Create/Manage assignments (no submission flow)

Module	Endpoints	Capabilities
Certificates	4 endpoints	Auto-issue, PDF download, Public verify
Community	11 endpoints	Posts, Comments, Likes — Backend only, NO frontend
Messaging	7 endpoints	Direct messages — Backend only, WebSocket not wired
Notifications	5 endpoints	CRUD, unread count — Frontend partially connected
Support Tickets	8 endpoints	Student + Admin ticket flow
Admin Panel	14 endpoints	Users, Courses, Analytics — partially connected
Instructor Tools	30+ endpoints	Earnings, Payouts, Students, Announcements
Profile & Settings	6 endpoints	Profile update, Avatar, Password, Preferences
Categories	7 endpoints	CRUD + image upload — Admin only create/edit

3. FEATURE AUDIT TABLE

All 47 features were audited by tracing the full flow from frontend UI action through Redux thunk, Axios API call, Spring Security filter, controller, service, repository, and database response.

■ Working	■ ■ Partial	■ Broken	■ Not Connected	■ UI Only
Feature	Module	Status	Severity	Notes / Issue
User Registration	Auth	■ Working	LOW	POST /auth/register → JWT tokens issued, email verification sent
Email Verification	Auth	■ Working	LOW	Token-based link sent via Gmail SMTP; /verify-email endpoint active
User Login	Auth	■ Working	LOW	JWT access + refresh tokens returned, stored in localStorage
Token Refresh	Auth	■ Working	LOW	Axios interceptor auto-calls /auth/refresh-token on 401
Logout	Auth	■ Working	LOW	Revokes refresh token server-side; clears Redux state
Forgot Password	Auth	■ Working	LOW	Email with reset link sent; token validated server-side
Reset Password	Auth	■ Working	LOW	POST /auth/reset-password with token and new password
Register Admin	Auth	■ ■ Partial	HIGH	/register-admin endpoint exists with NO auth guard — should be disabled in prod
Browse Courses	Courses	■ Working	LOW	GET /courses with filters/pagination; public, no auth required
Course Detail Page	Courses	■ Working	LOW	GET /courses/{id} and /courses/slug/{slug} both work
Course Curriculum	Courses	■ Working	LOW	GET /courses/{id}/curriculum returns modules + lessons
Course Search & Filter	Courses	■ Working	LOW	Filter by category, level, price, language supported in backend
Popular / Featured	Courses	■ Working	LOW	GET /courses/popular and /courses/featured-instructors functional
Enroll in Course	Courses	■ Working	LOW	POST /courses/{id}/enroll; checks subscription/payment status
Course Reviews	Courses	■ Working	LOW	CRUD reviews per course; verified purchase check in backend
Wishlist	Courses	■ Working	LOW	Add/remove/list wishlist; unique constraint enforced in DB
Cart & Checkout	Courses	■ ■ Partial	MEDIUM	Cart UI exists; checkout calls /payments/course/{id} but error handling is incomplete

Feature	Module	Status	Severity	Notes / Issue
Video Playback (Mux)	Lessons	■ ■ Partial	HIGH	Mux player integrated; signed URL obtained from /lessons/{id}/video-url. Requires MUX_TOKEN_ID env var — falls back to local without error in UI
Local Video Fallback	Lessons	■ Working	LOW	Graceful degradation to /files/ path when Mux not configured
Lesson Progress Tracking	Lessons	■ Working	LOW	POST /lessons/{id}/progress; tracks seconds watched + completion
Lesson Completion	Lessons	■ Working	LOW	POST /lessons/{id}/complete; updates enrollment progress %
Video Upload (Instructor)	Lessons	■ ■ Partial	HIGH	Mux direct upload URL created; frontend uploader present but upload flow has console.log debugging — unclear if error states are handled
Instructor Dashboard	Instructor	■ Working	LOW	GET /instructor/dashboard returns KPIs; charts rendered via ApexCharts
Create Course	Instructor	■ Working	MEDIUM	Multi-step form; POST /instructor/courses — extensive console.log in add-new-course form
Edit / Delete Course	Instructor	■ Working	LOW	PUT and DELETE /instructor/courses/{id} both implemented
Publish Course	Instructor	■ Working	LOW	POST /instructor/courses/{id}/publish — triggers admin review queue
Module & Lesson CRUD	Instructor	■ Working	LOW	Full CRUD for modules and lessons via /instructor/modules and /instructor/lessons
Course Thumbnail Upload	Instructor	■ Working	LOW	POST /instructor/courses/{id}/thumbnail — multipart upload
Announcements	Instructor	■ Working	LOW	CRUD announcements per instructor; GET /instructor/announcements
Quiz Creation	Instructor	■ Working	LOW	Full CRUD + publish/unpublish; questions and options managed
Assignments	Instructor	■ ■ Partial	MEDIUM	Backend CRUD exists; no student submission/grading flow implemented
Earnings & Statements	Instructor	■ Working	LOW	GET /instructor/earnings and /earnings/summary both connected
Payout Requests	Instructor	■ Working	LOW	POST /instructor/payouts/request; bank account info captured
Student Management	Instructor	■ Working	LOW	GET /instructor/students and per-course student lists
Instructor Certificates	Instructor	■ Working	LOW	GET /instructor/certificates — see issued certificates
Instructor Routes Bug	Instructor	■ Broken	HIGH	6 routes (wishlist, reviews, quiz attempts, orders, chat, referral) all point to /instructor/instructor-withdraw in all_routes.tsx
Student Dashboard	Student	■ Working	LOW	Stats, recent enrollments, progress — all connected

Feature	Module	Status	Severity	Notes / Issue
My Courses	Student	■ Working	LOW	GET /courses/my-courses; lists enrolled courses with progress
Student Quiz	Student	■ Working	LOW	Start/answer/submit quiz; score calculated server-side
Student Certificates	Student	■ Working	LOW	Auto-issued on course completion; PDF download functional
Order History	Student	■ Working	LOW	GET /payments/history — payment transactions listed
Student Tickets	Student	■ Working	LOW	Create, reply, close support tickets
Admin Dashboard	Admin	■ Working	LOW	GET /admin/dashboard returns platform statistics
User Management	Admin	■ Working	LOW	List, view, ban, unban, delete users — all endpoints connected
Course Approval	Admin	■ Working	LOW	GET /admin/pending-courses; approve/reject via POST endpoints
Transaction Management	Admin	■ Working	LOW	GET /admin/transactions — paginated payment history
Admin Analytics	Admin	■ Not Connected	HIGH	Admin analytics/reports/subscriptions routes defined in all_routes.tsx but NOT wired in router.link.tsx — pages unreachable
Course Payment	Payments	■ Working	MEDIUM	PayZone integration; payment URL generated and returned to frontend
Payment Webhook	Payments	■ Working	LOW	HMAC-SHA256 signature verification on webhook receipt
Community Posts	Community	■ Not Connected	HIGH	Backend fully implemented (11 endpoints); NO frontend pages or routes defined
Direct Messaging	Messaging	■ Not Connected	HIGH	Backend 7 endpoints exist; WebSocket configured server-side. Frontend WS_URL env var defined but NO WebSocket client implemented
Subscription Purchase	Subscriptions	■■ Partial	MEDIUM	POST /subscriptions/subscribe connected; subscription UI pages incomplete — missing subscription management dashboard

■ 4. DETAILED BUG REPORTS

BUG-001	Gmail App Password Hardcoded in Source Code	CRITICAL
Location	academy-backend/src/main/resources/application.yml — Line 45 · spring.mail.password	
Steps to Reproduce	1. Clone the repository 2. Open application.yml 3. Read spring.mail.password field	
Expected Result	Gmail credentials stored securely as environment variable only	
Actual Result	Real Gmail App Password "rzjr fkul oswn mifu" committed in plain text. This credential has likely been cached in git history.	
Suggested Fix	Immediately revoke this Gmail App Password via Google Account settings. Remove hardcoded value, use only \${MAIL_PASSWORD} with no default. Audit git history for other exposed secrets. Consider using git-secret or HashiCorp Vault.	

BUG-002	JWT Secret Key Committed to Source Control	CRITICAL
Location	academy-backend/src/main/resources/application.yml — app.jwt.secret	
Steps to Reproduce	1. Read application.yml 2. Observe: jwt.secret: \${JWT_SECRET:CakeDesignAcademySecretKeyForJWTTokenGen2024SuperSecure!}	
Expected Result	JWT secret must never appear in source code or git history	
Actual Result	Anyone with repository access can forge valid JWT tokens and impersonate any user, including admins.	
Suggested Fix	Rotate the JWT secret immediately. Remove the default value. All active sessions will be invalidated — this is acceptable for a security fix. Use a cryptographically random 512-bit key stored only in environment.	

BUG-003	CORS Wildcard + Credentials = Security Vulnerability	CRITICAL
Location	academy-backend/.../config/SecurityConfig.java — corsConfig setup	
Steps to Reproduce	1. Send request with Origin: https://evil.com and include credentials 2. Observe Access-Control-Allow-Origin: * with Access-Control-Allow-Credentials: true	
Expected Result	CORS should allow specific trusted origins only, or disable credentials with wildcard	
Actual Result	setAllowedOriginPatterns("*") combined with setAllowCredentials(true) is a CORS misconfiguration. Modern browsers block this combination, but some clients may not.	
Suggested Fix	Replace wildcard with explicit origins: [http://localhost:3000, https://yourdomain.com]. Never combine wildcard origins with credentials=true.	

BUG-004 Spring Actuator Endpoints Publicly Accessible		CRITICAL
Location	academy-backend/.../config/SecurityConfig.java — /actuator/** permits all	
Steps to Reproduce	1. Start the backend server 2. GET http://localhost:8080/actuator/health 3. GET http://localhost:8080/actuator/metrics	
Expected Result	Actuator endpoints should require ADMIN role	
Actual Result	Health, metrics, info, and other actuator endpoints are publicly accessible. Metrics can expose internal application state.	
Suggested Fix	In SecurityConfig: change /actuator/** from permitAll() to hasRole('ADMIN'). Alternatively disable unneeded actuator endpoints in application.yml.	

BUG-005 Unprotected Admin Registration Endpoint		CRITICAL
Location	academy-backend/.../controller/AuthController.java — POST /api/v1/auth/register-admin	
Steps to Reproduce	1. POST /api/v1/auth/register-admin with any valid user payload 2. No authentication required	
Expected Result	Admin registration must require existing ADMIN authentication	
Actual Result	Any anonymous user can call /register-admin to create an admin account. While there is a profile check, the endpoint is fully public.	
Suggested Fix	Add @PreAuthorize("hasRole('ADMIN')") or require ADMIN JWT. Better: remove the endpoint entirely and manage admin creation via database migration.	

BUG-006 Known Admin Credentials in Flyway Migration		CRITICAL
Location	academy-backend/src/main/resources/db/migration/V2__quiz_tables.sql	
Steps to Reproduce	1. Read V2__quiz_tables.sql 2. Find INSERT for initial admin user with password hash for Admin@1234	
Expected Result	Admin credentials seeded via migration must be random and require immediate change	
Actual Result	Admin user seeded with well-known password Admin@1234. This will be applied to every new database deployment.	
Suggested Fix	Remove the hardcoded password. Generate a strong random password per deployment. Force password change on first login for seeded admin accounts.	

BUG-007	Database Default Password "0000"	CRITICAL
Location	academy-backend/src/main/resources/application.yml — spring.datasource.password	
Steps to Reproduce	1. Read application.yml 2. Observe: password: \${DB_PASSWORD:0000}	
Expected Result	No default password — require explicit environment configuration	
Actual Result	If DB_PASSWORD env var is not set, the application connects with password "0000". This trivially weak default would be used if the env var is forgotten.	
Suggested Fix	Remove the default value entirely. Application should fail to start rather than connect with a weak default.	

BUG-008	Mux Public Env Key Hardcoded in Frontend .env	CRITICAL
Location	template/.env — REACT_APP_MUX_ENV_KEY=0ob8u9r0velabi6jsfv8b3qu	
Steps to Reproduce	1. Read template/.env file 2. Observe hardcoded Mux environment key	
Expected Result	Mux public key should be loaded from secure environment configuration, not committed to git	
Actual Result	Mux environment key is committed to the repository. While public keys have limited scope, rotation is best practice.	
Suggested Fix	Add .env to .gitignore, use .env.example as template. Rotate the Mux key via the Mux dashboard.	

BUG-009	6 Instructor Routes All Point to Wrong Path	HIGH
Location	template/src/feature-module/router/all_routes.tsx — Lines 42-47	
Steps to Reproduce	1. Log in as INSTRUCTOR 2. Navigate to any of: instructor wishlist, reviews, quiz attempts, orders, chat, or referral pages 3. All redirect to /instructor/instructor-withdraw	
Expected Result	Each route should navigate to its dedicated page	
Actual Result	instructorWishlist, instructorReviews, instructorQuizAttempts, instructorOrders, instructorChat, instructorReferral are ALL mapped to /instructor/instructor-withdraw. Clicking any of these 6 menu items opens the withdraw/payout page.	
Suggested Fix	Assign each route a unique, correct path in all_routes.tsx. Verify corresponding component files exist or create them.	

BUG-010 Community Module: Backend Complete, Zero Frontend		HIGH
Location	template/src/feature-module/ — No community pages exist	
Steps to Reproduce	1. Inspect backend: 11 community endpoints exist (posts, comments, likes) 2. Inspect frontend: no /community/* routes in router.link.tsx 3. No community service connected to UI	
Expected Result	Community features should be accessible to authenticated users	
Actual Result	The entire community module (forum, posts, discussions) is fully implemented in the backend with 11 endpoints but has no frontend implementation. Users cannot access this feature.	
Suggested Fix	Implement community frontend pages: post listing, post detail, create post, comments. Wire to existing community.service.ts. Add routes to router.link.tsx.	

BUG-011 WebSocket Messaging Not Implemented Client-Side		HIGH
Location	template/src/.env and template/src/environment.tsx	
Steps to Reproduce	1. Check REACT_APP_WS_URL is defined 2. Search codebase for WebSocket client implementation 3. Find none exists	
Expected Result	Real-time messaging should connect via WebSocket after login	
Actual Result	WS_URL is configured in environment (ws://localhost:8080/ws), backend WebSocket is configured with STOMP, but there is no WebSocket client code anywhere in the frontend. The messaging UI exists but likely polls or does nothing.	
Suggested Fix	Implement STOMP WebSocket client using @stomp/stompjs or sockjs-client. Connect on user login, subscribe to personal message channels.	

BUG-012 Admin Analytics Pages Defined but Not Routed		HIGH
Location	template/src/feature-module/router/all_routes.tsx and router.link.tsx	
Steps to Reproduce	1. Define adminAnalytics, adminReports, adminSubscriptions routes in all_routes.tsx 2. Check router.link.tsx — these routes are absent 3. Navigating to these URLs returns 404	
Expected Result	Admin analytics, reports, and subscriptions pages should be accessible	
Actual Result	Routes adminSubscriptions, adminAnalytics, adminReports exist in all_routes.tsx but are NOT imported or rendered in router.link.tsx. Backend endpoints for analytics exist (/admin/analytics) but are unreachable from frontend.	
Suggested Fix	Add the missing route entries to router.link.tsx with their corresponding component imports.	

BUG-013 Console.log Statements Throughout Production Code		MEDIUM
Location	template/src/services/api/axios.config.ts, course.service.ts, admin.service.ts, add-newCourse.tsx	
Steps to Reproduce	1. Open browser DevTools console 2. Perform any API call (login, load courses, submit form) 3. Observe API responses logged in plain text	
Expected Result	No console output in production — API responses may contain PII	
Actual Result	API responses (including user data, tokens, course details) are logged to console in production build. This exposes sensitive data in browser devtools.	
Suggested Fix	Remove all console.log/debug statements. Use a proper logging library (e.g., loglevel) with WARN/ERROR level in production. Add ESLint rule: no-console.	

BUG-014 TypeScript Unsafe any Cast in RoleGuard		MEDIUM
Location	template/src/feature-module/common/RoleGuard.tsx — Line 19	
Steps to Reproduce	1. Read RoleGuard.tsx 2. Observe: user.role as any 3. Type safety for authorization guard bypassed	
Expected Result	RoleGuard should use strongly-typed UserRole enum for comparisons	
Actual Result	Using as any in the authorization guard can mask type mismatches between UserRole enum values and comparison strings, potentially causing authorization bypass in edge cases.	
Suggested Fix	Import UserRole enum. Replace as any with proper typed comparison: (user.role as UserRole) === UserRole.ADMIN, or use type guards.	

BUG-015 Build Artifacts Committed to Git Repository		MEDIUM
Location	template/build/, template/build_output.txt, template/build_output2.txt, template/ts_errors.txt	
Steps to Reproduce	1. Run git status 2. Observe: build/ directory, build_output files, ts_errors.txt (272KB) committed	
Expected Result	Build artifacts should be in .gitignore — only source code committed	
Actual Result	272KB ts_errors.txt and full /build directory are committed. This pollutes the repository, inflates clone size, and may expose internal type errors.	
Suggested Fix	Add build/, *.txt (build artifacts), build_output*.txt to .gitignore. Run: git rm -r --cached build/ build_output*.txt ts_errors.txt tmp_update_routes.js. Commit the .gitignore changes.	

BUG-016 Assignment Submission Flow Missing		MEDIUM
Location	academy-backend/.../controller/AssignmentController.java and frontend instructor-assignment page	
Steps to Reproduce	1. Create an assignment as instructor 2. Attempt to submit as student 3. No student submission endpoint exists	
Expected Result	Students should be able to submit and receive grades for assignments	
Actual Result	The Assignment entity and CRUD endpoints exist for instructors, but there are no student submission endpoints, no submission entity, and no grading flow. Assignments are essentially read-only for students.	
Suggested Fix	Add StudentAssignmentSubmission entity, repository, and endpoints: POST /student/assignments/{id}/submit, GET /student/assignments/{id}/submission. Add grading endpoint for instructors.	

BUG-017 Missing Route Slash: pages/contact-us		LOW
Location	template/src/feature-module/router/all_routes.tsx — contactUs route definition	
Steps to Reproduce	1. Check all_routes.tsx contactUs value 2. Observe: "pages/contact-us" (missing leading /) 3. Navigation to contact page produces unexpected behavior	
Expected Result	Route should be "/pages/contact-us" with leading slash	
Actual Result	Route is defined as "pages/contact-us" instead of "/pages/contact-us". This causes relative path resolution issues when navigating from nested routes.	
Suggested Fix	Add leading slash: contactUs: '/pages/contact-us'	

BUG-018 Subscription Access Guard Incomplete		MEDIUM
Location	template/src/ — SubscriptionGuard or premium access logic	
Steps to Reproduce	1. Enroll in a premium course without active subscription 2. Attempt to access premium lesson video	
Expected Result	Premium course content should be blocked without active subscription or payment	
Actual Result	Backend has subscription status checks, but frontend SubscriptionGuard only checks authentication, not actual subscription validity. Premium content may be accessible without payment.	
Suggested Fix	Enhance access guard: check user.subscription.status === ACTIVE or course.enrollmentType === PAID before allowing lesson access. Backend already enforces this but frontend should provide proper UX feedback.	

BUG-019 Multiple Home Page Variants Only One Active		LOW
Location	template/src/feature-module/HomePages/ — home-one through home-six	
Steps to Reproduce	1. Navigate to / 2. homeone is rendered 3. home-two through home-six are unreachable without route changes	
Expected Result	Only one home page variant should exist, or a switcher should be provided	
Actual Result	Six distinct home page implementations exist (homeone–homeSix). Only homeone is routed. The other five are dead code occupying approximately 600 lines each. This creates maintenance overhead.	
Suggested Fix	Remove home-two through home-six if they are not needed. If A/B testing, use a proper feature flag system.	

BUG-020 tmp_update_routes.js Temporary Script in Repository		LOW
Location	template/tmp_update_routes.js	
Steps to Reproduce	1. View repository root 2. Observe tmp_update_routes.js — a temporary development script	
Expected Result	No temporary scripts should exist in the repository	
Actual Result	A temporary JavaScript file used during development is committed to the repository. This suggests sloppy git hygiene and may contain executable code.	
Suggested Fix	Delete the file: git rm tmp_update_routes.js. Update .gitignore to exclude tmp_*.js patterns.	

■ 5. BACKEND CONNECTIVITY REPORT

This section maps each frontend service call to its corresponding backend endpoint, identifying connection status, mismatches, and missing integrations.

Frontend Call	Backend Endpoint	Status	Notes
authService.login()	POST /api/v1/auth/login	■ Connected	Returns access+refresh tokens
authService.register()	POST /api/v1/auth/register	■ Connected	Email verification sent
authService.logout()	POST /api/v1/auth/logout	■ Connected	Revokes refresh token
authService.forgotPassword()	POST /api/v1/auth/forgot-password	■ Connected	Reset email sent
authService.resetPassword()	POST /api/v1/auth/reset-password	■ Connected	Password updated
courseService.getCourses()	GET /api/v1/courses	■ Connected	Paginated, filterable
courseService.enrollInCourse()	POST /api/v1/courses/{id}/enroll	■ Connected	Checks payment/subscription
courseService.getMyEnrolledCourses()	GET /api/v1/courses/my-courses	■ Connected	Returns enrollment list
courseService.addToWishlist()	POST /api/v1/courses/{id}/wishlist	■ Connected	Unique constraint protected
courseService.getCategories()	GET /api/v1/categories	■ Connected	Console.log present
instructorService.getDashboard()	GET /api/v1/instructor/dashboard	■ Connected	KPIs returned
instructorService.createCourse()	POST /api/v1/instructor/courses	■ Connected	Slug auto-generated
instructorService.uploadLessonVideo()	POST /api/v1/instructor/lessons/{id}/upload-video	■ Connected	Creates Mux upload URL
instructorService.requestPayout()	POST /api/v1/instructor/payouts/request	■ Connected	Bank details captured
instructorService.getEarnings()	GET /api/v1/instructor/earnings	■ Connected	Paginated earnings
adminService getUsers()	GET /api/v1/admin/users	■ Connected	Paginated + searchable
adminService.banUser()	POST /api/v1/admin/users/{id}/ban	■ Connected	Reason captured
adminService.approveCourse()	POST /api/v1/admin/courses/{id}/approve	■ Connected	Updates course status
adminService.getTransactions()	GET /api/v1/admin/transactions	■ Connected	Full transaction history

Frontend Call	Backend Endpoint	Status	Notes
adminService.getAnalytics()	GET /api/v1/admin/analytics	■ NOT CONNECTED	Route missing from router.link.tsx — endpoint exists but unreachable
adminService.getReports()	GET /api/v1/admin/reports	■ NOT CONNECTED	Route missing from router.link.tsx
paymentService.purchaseCourse()	POST /api/v1/payments/course/{id}	■ Connected	Returns PayZone payment URL
paymentService.getPaymentHistory()	GET /api/v1/payments/history	■ Connected	Transaction list
N/A — No frontend	GET /api/v1/community/posts	■ NOT CONNECTED	Community module has no frontend pages
N/A — No frontend	POST /api/v1/community/posts	■ NOT CONNECTED	No create post UI
N/A — No WebSocket client	WebSocket /ws/chat	■ NOT CONNECTED	STOMP server configured, no JS client
N/A — No frontend	GET /api/v1/subscriptions/my-subscription	■ Partial	Subscription UI incomplete

■ 6. UI/UX ISSUES

Multiple Home Page Variants (6)	HIGH	Six home page implementations exist (homeone–homeSix). Only homeone is routed. The dead code creates confusion and maintenance overhead. Users never see the other 5 designs.
Instructor Dashboard: 6 Broken Navigation Links	CRITICAL	Instructor sidebar links for Wishlist, Reviews, Quiz Attempts, Orders, Chat, and Referral all navigate to the Payout/Withdraw page. Users are confused when clicking these menu items.
Admin Section Missing Pages	HIGH	Admin Analytics, Admin Reports, and Admin Subscriptions menu items are present in the UI but navigate to a blank/404 page since the routes are not wired in the router.
No Error Boundary Components	MEDIUM	No React error boundaries are implemented. Any uncaught JavaScript error in a component will crash the entire application rather than showing a graceful error message.
Community Feature Invisible to Users	HIGH	The community/forum feature is fully built on the backend but completely invisible in the UI. There is no navigation link, no page, and no way for users to discover or use it.
Real-time Messaging UI Without Real-time	HIGH	The messaging UI pages exist for both students and instructors, but without a WebSocket client, messages either require manual refresh or do not update in real-time.
Subscription Management UX Incomplete	MEDIUM	Users can initiate a subscription but cannot view their current plan details, manage billing, or see clear next-billing-date information in a dedicated UI.
Assignment Feature: Student View Missing	MEDIUM	Instructors can create assignments but students have no UI to view, submit, or receive grades for assignments. The feature is half-implemented from a user journey perspective.
Cart/Checkout Error States Not Handled	MEDIUM	When a payment fails or PayZone returns an error, the checkout page does not show a meaningful error message. Users are left in an unclear state.
Console Logs Visible in DevTools	LOW	API responses including user data, course details, and auth tokens are logged to the browser console in production, exposing sensitive data to tech-savvy users.
Inconsistent Loading States	LOW	Some pages show loading spinners while data fetches, others show blank screens. No unified skeleton-loading pattern across the application.
Blog Pages: Static Template Only	LOW	8 blog layout variants exist as static templates with no backend connection. No blog API endpoints exist. Blog links in navigation lead to dummy content.

7. PERFORMANCE ANALYSIS

Bundle Size: 45+ Production Dependencies		MEDIUM
Problem	The frontend includes Ant Design (5.x), Bootstrap, PrimeReact, Swiper, React Slick, and multiple icon sets simultaneously. This creates a very large initial JavaScript bundle affecting Time to Interactive (TTI).	
Fix	Implement code splitting per route. Use tree-shaking aggressively. Replace multiple UI libraries with a single consistent one. Target bundle <500KB gzipped.	
Six Unused Home Page Variants		LOW
Problem	Five of six home page implementations are never rendered but are still included in the bundle. Each contains Swiper carousels, animation libraries, and multiple components.	
Fix	Delete unused home page variants. The dead code wastes compile time and bundle space.	
No React.memo or useMemo on Heavy Components		MEDIUM
Problem	Dashboard components re-render on every Redux state change. Course listing cards, charts (ApexCharts), and sidebar components likely re-render unnecessarily.	
Fix	Wrap stable components with React.memo. Memoize expensive selectors with useMemo/reselect. Profile with React DevTools Profiler.	
Synchronous Axios Config: Console Logs on Every Request		LOW
Problem	Every API request logs to console via the interceptor. This is synchronous I/O on the critical request path.	
Fix	Remove all console.log calls from the axios interceptor.	
HikariCP: 10-Connection Pool May Be Insufficient		MEDIUM
Problem	maximum-pool-size: 10 with minimum-idle: 5. For a platform expecting concurrent students watching videos and submitting quizzes, 10 connections may cause queuing under load.	
Fix	Monitor pool utilization in production. Consider increasing to 20-30 based on DB server capacity. Enable HikariCP metrics via Actuator.	
Hibernate show-sql: true in Non-Production Logs		LOW
Problem	show-sql: false is set, but logging.level.org.hibernate.SQL: DEBUG is enabled — this logs all SQL to the console in DEBUG mode, creating I/O overhead.	
Fix	Set org.hibernate.SQL to INFO or WARN in production profile. Only enable SQL logging in development.	

No Caching Layer for Public Course Data		MEDIUM
Problem	GET /courses, /courses/popular, /categories — these are called on every page load by every user with no caching. Under load, this hits the database on every request.	
Fix	Add Spring Cache (@Cacheable) on popular course and category queries. Use Redis or Caffeine as backing store. TTL of 60-300 seconds for public data.	

Video Loading: No CDN or Pre-signed URL Caching		MEDIUM
Problem	Mux signed URLs (1-hour expiry) are fetched per lesson view. No client-side caching of recently obtained URLs. Each video start triggers an API call.	
Fix	Cache signed URLs in React state or localStorage with expiry. Implement prefetching for next lesson URL while watching current lesson.	

■ 8. SECURITY RISK ASSESSMENT

Security assessment based on OWASP Top 10 (2021) and general web application security best practices. Eight critical vulnerabilities require immediate remediation before any production deployment.

Vulnerability	Location	Risk	Recommendation
Gmail App Password in Source Code	application.yml (MAIL_PASSWORD default)	CRITICAL	Revoke immediately. Remove default. Store only in vault/env variable.
JWT Secret Key in Source Control	application.yml (JWT_SECRET default)	CRITICAL	Rotate secret. Remove default value. Invalidate all existing sessions.
CORS: Wildcard + Credentials=True	SecurityConfig.java — corsConfig	CRITICAL	Replace * with explicit trusted origins list.
Spring Actuator Publicly Accessible	SecurityConfig.java — /actuator/**	CRITICAL	Require ADMIN role for all actuator endpoints.
Unauthenticated Admin Registration Endpoint	AuthController — /auth/register-admin	CRITICAL	Add @PreAuthorize('ADMIN') or remove the endpoint entirely.
Known Admin Password in DB Migration	V2__quiz_tables.sql — Admin@1234	CRITICAL	Remove seeded password. Force password change on first login.
Database Default Password "0000"	application.yml (DB_PASSWORD default)	CRITICAL	Remove default. Fail startup if DB_PASSWORD not set.
Mux API Key in .env Committed to Git	template/.env	CRITICAL	Add .env to .gitignore. Rotate key. Use .env.example.
JWT Stored in localStorage (XSS Risk)	auth.service.ts — localStorage.getItem(ACCESS_TOKEN_KEY)	HIGH	Consider HttpOnly cookies for token storage. Add CSP headers.
No Rate Limiting on Auth Endpoints	/auth/login, /auth/register, /auth/forgot-password	HIGH	Add Spring Boot rate limiting (Bucket4j or custom filter) on auth routes.
No Input Validation on Course/Profile Fields	Backend controllers — missing @Valid annotations on some endpoints	HIGH	Add @Valid + Bean Validation (javax.validation) to all request DTOs.
Public Webhook Endpoints Without IP Filtering	/payments/webhook, /subscriptions/webhook	MEDIUM	In addition to signature verification, whitelist PayZone IP ranges.
Sensitive Data in Console Logs	axios.config.ts, admin.service.ts	MEDIUM	Remove all console.log statements. Implement server-side audit logging.
IDOR: No Ownership Validation in Some Endpoints	InstructorController — course/module/lesson endpoints	HIGH	Verify authenticated instructor owns the course before modifying modules/lessons. Check service implementations for ownership validation.
Swagger UI Publicly Accessible	SecurityConfig — /swagger-ui/**, /v3/api-docs/**	MEDIUM	Disable Swagger in production (springdoc.swagger-ui.enabled: false). Or require ADMIN role.

OWASP Top 10 Mapping:

OWASP Category	Risk Level	Finding
A01 — Broken Access Control	MEDIUM	Potential IDOR in instructor endpoints; admin registration endpoint unprotected

OWASP Category	Risk Level	Finding
A02 — Cryptographic Failures	CRITICAL	JWT secret + Gmail password in plaintext source code
A03 — Injection	LOW	Spring Data JPA parameterized queries prevent SQL injection
A04 — Insecure Design	HIGH	Admin seeded with known credentials; no rate limiting design
A05 — Security Misconfiguration	CRITICAL	Actuator public; CORS wildcard+credentials; Swagger UI public
A06 — Vulnerable Components	LOW	Dependencies current as of audit date; monitor for CVEs
A07 — Auth & Session Failures	HIGH	JWT in localStorage vulnerable to XSS; no rate limiting on login
A08 — Software Integrity	LOW	No SBOM or integrity checks; package-lock.json committed
A09 — Logging Failures	MEDIUM	Sensitive data logged to console; no centralized audit log
A10 — SSRF	LOW	External API calls to Mux/PayZone use configured base URLs; low risk

9. RECOMMENDATIONS & FIX PRIORITY PLAN

Issues are organized into three sprints. Critical security issues must be resolved before any production deployment. Feature completeness and UX improvements follow in subsequent sprints.

SPRINT 0 — IMMEDIATE (Before Production)

ID	Priority	Task	Effort
SEC-1	CRITICAL	Revoke Gmail app password; remove hardcoded MAIL_PASSWORD default from application.yml	1 hour
SEC-2	CRITICAL	Rotate JWT secret; remove hardcoded JWT_SECRET default; invalidate all sessions	1 hour
SEC-3	CRITICAL	Fix CORS: replace allowedOriginPatterns("*") with explicit trusted origins	30 min
SEC-4	CRITICAL	Restrict /actuator/** to ADMIN role in SecurityConfig	15 min
SEC-5	CRITICAL	Add @PreAuthorize("hasRole('ADMIN')") to /register-admin or delete the endpoint	15 min
SEC-6	CRITICAL	Remove seeded admin password from V2 migration; add first-login password change	2 hours
SEC-7	CRITICAL	Remove DB_PASSWORD default "0000"; add startup validation	15 min
SEC-8	CRITICAL	Add template/.env to .gitignore; rotate Mux env key	30 min
GIT-1	HIGH	Remove build/, build_output*.txt, ts_errors.txt, tmp_update_routes.js from git	30 min

SPRINT 1 — HIGH PRIORITY (Week 1–2)

ID	Priority	Task	Effort
FE-1	HIGH	Fix 6 duplicate instructor routes in all_routes.tsx pointing to instructor-withdraw	1 hour
FE-2	HIGH	Wire admin analytics, reports, subscriptions routes in router.link.tsx	2 hours
FE-3	HIGH	Remove all console.log statements from production code (axios interceptor, services, forms)	3 hours
FE-4	HIGH	Fix missing leading slash: contactUs route in all_routes.tsx	15 min
FE-5	HIGH	Implement React error boundary components for graceful error handling	4 hours
BE-1	HIGH	Add rate limiting on /auth/login, /auth/register, /auth/forgot-password (Bucket4j)	4 hours
BE-2	HIGH	Disable Swagger UI in production profile (springdoc.swagger-ui.enabled: false)	15 min
BE-3	HIGH	Verify ownership validation in InstructorController for all course/module/lesson mutations	4 hours
FE-6	HIGH	Fix TypeScript as any cast in RoleGuard.tsx — use typed UserRole enum	1 hour

SPRINT 2 — FEATURE COMPLETION (Week 3–4)

ID	Priority	Task	Effort
FEAT-1	HIGH	Implement Community frontend: post list, post detail, create post, comments	3 days

ID	Priority	Task	Effort
FEAT-2	HIGH	Implement WebSocket client (STOMP/sockjs) for real-time messaging	2 days
FEAT-3	HIGH	Complete subscription management UI: current plan, billing date, cancel flow	1 day
FEAT-4	MEDIUM	Implement student assignment submission flow + instructor grading UI	2 days
FEAT-5	MEDIUM	Improve cart/checkout error handling with PayZone failure states	1 day
FEAT-6	MEDIUM	Complete video upload UX: remove console.logs, add proper error states, upload progress	1 day
FE-7	MEDIUM	Remove 5 unused home page variants (home-two through home-six)	2 hours
FE-8	MEDIUM	Add unified skeleton-loading pattern across all data-fetch pages	1 day

SPRINT 3 — PERFORMANCE & POLISH (Week 5–6)

ID	Priority	Task	Effort
PERF-1	MEDIUM	Implement route-based code splitting to reduce initial bundle size	1 day
PERF-2	MEDIUM	Consolidate UI libraries: pick one primary (Ant Design or Bootstrap, not both)	2 days
PERF-3	MEDIUM	Add Spring Cache (@Cacheable) on public course/category queries with Redis/Caffeine	1 day
PERF-4	MEDIUM	Add React.memo to stable components (CourseCard, Sidebar, DashboardStat)	1 day
PERF-5	LOW	Increase HikariCP pool size based on production load testing results	2 hours
PERF-6	LOW	Cache Mux signed URLs client-side to avoid re-fetching on each lesson load	4 hours
SEC-9	MEDIUM	Migrate JWT from localStorage to HttpOnly cookies to mitigate XSS token theft	2 days
SEC-10	LOW	Add PayZone IP allowlist in addition to signature verification on webhook endpoints	4 hours

Final Note: The Cake Design Academy platform has a strong architectural foundation with comprehensive backend coverage and a well-organized frontend. The core learning flow (browse → enroll → watch → quiz → certificate) is functional end-to-end. The primary concerns are: (1) security secrets in source code requiring immediate rotation, (2) several significant features built on the backend but missing frontend implementations, and (3) routing bugs that break navigation for instructors. With the Sprint 0 and Sprint 1 fixes applied, the platform would be ready for a secure beta deployment. Full production readiness requires Sprint 2 completion.